Python - Final Project

תיאור הפרויקט

בנו שרת FastAPI המנתח את ביצועי צוות עבודה. מקור הנתונים הוא השרת החיצוני jsonplaceholder.typicode.com, בנתיבים /users ו-/todos. השרת יאפשר לצפות בניתוח הביצועים, לנהל משימות חדשות ולשאול שאלות על הנתונים באמצעות מודל שפה. אין להשתמש במסד נתונים: הנתונים יישמרו בזיכרון ובקבצי JSON ו-CSV בלבד.

שלב 1: הקמת הפרויקט (15 דקות)

צרו את הקבצים main.py (נתיבים, middleware וטיפול בחריגות), data_service.py (משיכת נתונים וניתוח), ai_service.py (מודל השפה והלוגים), requirements.txt, קובץ env. וקובץ gitignore.. טענו את המפתחות ANTHROPIC_API_KEY ו-APP_API_KEY בעזרת load_dotenv() ו-os.environ.get(). אם מפתח חסר, השרת יציג הודעה ברורה וימשיך לפעול.

שלב 2: משיכת נתונים (25 דקות)

כתבו פונקציה fetch_json(path) הפונה לשרת עם timeout=5 ו-raise_for_status(), ומחזירה None במקרה של RequestException. בנו DataFrame של עובדים הכולל user_id, name, email ו-city (העיר נמצאת תחת address), באמצעות list comprehension, ו-DataFrame של משימות. ודאו שהעמודה completed מסוג bool. שמרו את הנתונים במשתנה גלובלי (cache). כאשר המקור אינו זמין, הנתיב יחזיר 503.

שלב 3: ניתוח נתונים (35 דקות)

שנו את שם העמודה userId ל-user_id ומזגו (merge) את המשימות עם העובדים. בעזרת groupby ו-agg חשבו לכל עובד את סך המשימות, מספר המשימות שהושלמו ואחוז ההשלמה (שתי ספרות אחרי הנקודה), ומיינו את התוצאה. חשבו ב-numpy ממוצע וסטיית תקן של אחוזי ההשלמה, ומצאו בעזרת boolean indexing את העובדים שמתחת לממוצע. בנו pivot_table של ערים מול סטטוס השלמה (aggfunc="count").

שלב 4: ניהול משימות – CRUD (35 דקות)

הגדירו מודל TaskIn (title, user_id, due_date, priority, done, notes אופציונלי) ומודל TaskOut ללא notes, והשתמשו ב-response_model. בדקו שהכותרת אינה ריקה לאחר strip(), שהעדיפות בין 1 ל-3 ושהעובד קיים, ואחרת החזירו 400. צרו חריגה TaskNotFoundError עם exception_handler המחזיר 404 והודעה מפורטת. כל שינוי יישמר ל-tasks.json, וטעינה בעליית השרת תטפל ב-FileNotFoundError. יצירת משימה תחזיר 201.

שלב 5: תשתית ואבטחה (20 דקות)

צרו Dependency לדפדוף (page, page_size) לכל נתיב שמחזיר רשימה, ו-Dependency הבודק את ה-X-API-Key Header מול APP_API_KEY ומחזיר 401 אם אינו תואם. הגנו בו על כל נתיב שמשנה נתונים. הוסיפו middleware המוסיף לכל תשובה X-Process-Time ומדפיס method, path ו-status_code, והגדירו CORS עבור http://localhost:5173 בלבד.

שלב 6: עוזר חכם מבוסס מודל שפה (40 דקות)

צרו נתיב POST /ai/ask המקבל prompt ו-conversation_id, ומחזיר 400 עבור פרומפט ריק או ארוך מ-500 תווים. בנו System Prompt דינמי מתוך תוצאות הניתוח (שלושת המובילים, הממוצע והעיר החזקה), ושמרו היסטוריית שיחה לכל conversation_id. בכל קריאה יירשמו ל-ai_logs.csv הזמן, אורך הפרומפט והסטטוס, באמצעות try / except / finally, גם במקרה של שגיאה. אין לשמור מפתחות או את תוכן הפרומפט. כאשר אין מפתח API, החזירו תשובה קבועה (Mock).

נתיבים נדרשים

/stats/users (עם מיון ודפדוף), /stats/summary, /stats/low-performers, /stats/cities, /tasks ו-/tasks/{task_id} בפעולות GET, POST, PUT ו-DELETE, /tasks/overdue (משימות שלא בוצעו ושמועדן עבר, כולל days_late בעזרת pd.to_datetime ו-dt.days), /ai/ask ו-/ai/logs/stats (אחוז הצלחה ואורך פרומפט ממוצע מתוך קובץ הלוגים). שימו לב: הנתיב /tasks/overdue חייב להיות מוגדר לפני /tasks/{task_id}.